

```
// -----
// -----
// GharKaKhana – Send Push Notification
// (Vercel serverless function)
// -----
// -----
// This is what makes a notification reach
// a customer even if they closed the
// site/app completely – a REAL Web Push,
// different from the in-app toasts
// used elsewhere (which only work while
// the site is actually open).
//
// SETUP NEEDED (one-time, see the chat
// message for full steps):
// 1. Save this file as api/send-
// notification.js in the same Vercel project
// as your existing api/chat.js.
// 2. On Vercel, add a "web-push"
// dependency (package.json) – Vercel installs
// it automatically on deploy, you
// don't run anything locally.
// 3. Add these Environment Variables in
// Vercel (same place as GROQ_API_KEY):
// VAPID_PUBLIC_KEY =
// BN1uvTKK3wCtscGGHikrYaleKyZjxrABKZ7L8RG-
// zpCjiwtYcyrOUaQS-
// AVbekgKV2xjQwFL1bWpRD0798078u8
// VAPID_PRIVATE_KEY =
// S0kDdaDIaN3XolNMmWrP28EowoUqc9tuK5vJn4u1POM
```

```

//      NOTIFY_SECRET      = (aap khud
koi bhi lambi random string banao – sirf
aapko pata ho)
//      FIREBASE_SERVICE_ACCOUNT = (poora
service account JSON, ek hi line mein –
Firebase console > Project Settings >
Service Accounts > Generate new private
key)
// 4. Update ALLOWED_ORIGINS below to
match your real site domain(s).
// -----
-----

const webpush = require("web-push");
const admin = require("firebase-admin");

const ALLOWED_ORIGINS = [
  "https://sadiqmalek818.github.io",
  "https://gharkakhana-kappa.vercel.app",
];

if (!admin.apps.length) {
  admin.initializeApp({
    credential:
admin.credential.cert(JSON.parse(process.en
v.FIREBASE_SERVICE_ACCOUNT)),
  });
}
const db = admin.firestore();

webpush.setVapidDetails(
  "mailto:admin@gharkakhana.example",
  process.env.VAPID_PUBLIC_KEY,
  process.env.VAPID_PRIVATE_KEY

```

```

);

export default async function handler(req,
res) {
  const origin = req.headers.origin || "";
  if (ALLOWED_ORIGINS.includes(origin))
res.setHeader("Access-Control-Allow-
Origin", origin);
  res.setHeader("Access-Control-Allow-
Methods", "POST, OPTIONS");
  res.setHeader("Access-Control-Allow-
Headers", "Content-Type");

  if (req.method === "OPTIONS") {
res.status(204).end(); return; }
  if (req.method !== "POST") {
res.status(405).json({ error: "POST use
karo" }); return; }

  const { secret, title, body, url } =
req.body || {};
  if (!secret || secret !==
process.env.NOTIFY_SECRET) {
    res.status(403).json({ error: "Galat
secret" });
    return;
  }
  if (!title || !body) {
    res.status(400).json({ error: "title
aur body dono chahiye" });
    return;
  }

  try {

```

```

    const snapshot = await
db.collection("pushSubscriptions").get();
    const payload = JSON.stringify({ title,
body, url: url || "/" });

    let sent = 0, failed = 0, removed = 0;
    await Promise.all(
        snapshot.docs.map(async (doc) => {
            const sub = doc.data();
            try {
                await
webpush.sendNotification(sub.subscription,
payload);
                sent++;
            } catch (err) {
                failed++;
                // 410/404 means that
subscription is dead (customer uninstalled,
                // cleared data, etc.) – clean it
up so future sends don't keep
                // retrying it.
                if (err.statusCode === 410 ||
err.statusCode === 404) {
                    await doc.ref.delete();
                    removed++;
                }
            }
        })
    );

    res.status(200).json({ sent, failed,
removed, total: snapshot.size });
    } catch (err) {
        console.error("send-notification

```

```
failed:", err);  
    res.status(500).json({ error: "Server  
error: " + err.message });  
  }  
}
```